

Session 6.2

How Agentic AI Works: A Loop Built From Scratch, the Harness, the Session, and the Team

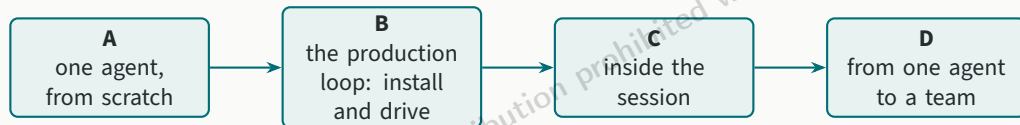
Part two of five: the loop in thirty lines you could write yourself, each of the four marks in code, the production harness that runs the same loop, what a session carries and costs, and how one agent becomes a team



Zhigang Feng

The Question, and the Route

How does agentic AI work — and could you write one yourself?



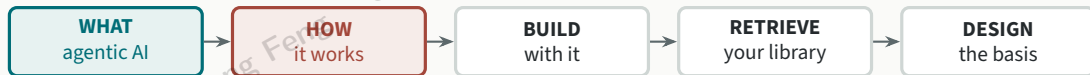
- ▶ A working definition: “An LLM agent runs tools in a loop to achieve a goal.”
- ▶ Deck 6.1 named four marks — tool use, autonomy, iteration, persistence. Part A puts each one into code; the running task is official uninsurance rates by age.
- ▶ **An agent is a model, a loop and tools; everything else is rules and files.**

Source: S. Willison, “I think ‘agent’ may finally have a widely enough agreed upon definition to be useful jargon now”, 18 Sep 2025; Miao & Feng, *AI for Economists*, ch. 15, §3.3.

Session 6.2

One Agent, From Scratch

One call, one loop, one tool, three rules, memory and a verifier: the machine in thirty lines



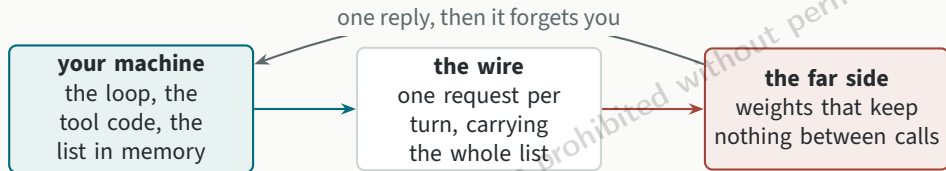
Step 0: The Simplest Model Call, and What It Cannot Do

```
messages = [  
    {"role": "user",  
     "content": "Plot uninsurance  
                by age, 2008–2024."}  
]  
reply = model(messages)  
print(reply.text)
```

- ▶ What comes back is *a description of what you should do*, possibly with code in it.
- ▶ No file was read. No command ran. Nothing on disk changed.
- ▶ **The model has no hands. Everything in this deck is about what happens when you give it some.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.1.

Step 1: The Loop, Not a Place



- ▶ Plan, act, observe, repeat: the model decides; the harness executes and reports back; the model reads the report and decides again.
- ▶ An error message is not a failure of the loop; it is the observation the next step is planned from.
- ▶ Two of Deck 6.1's four marks live here: autonomy and iteration.
- ▶ **The loop stops when the model stops asking for tools — not when the task is correct.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3.

Step 2: A Tool Is a Typed Function

```
{ "name": "score_forecast",  
  "description":  
    "Score a forecast of the  
    ten-year yield on the  
    frozen test window.",  
  "input_schema": {  
    "type": "object",  
    "properties": {  
      "path": { "type": "string" },  
      "metric": { "enum": [ "rmse",  
                           "mae" ] } },  
    "required": [ "path" ] } }
```

- ▶ A tool is a function the model may call: a name, a schema for its arguments, a description.
- ▶ The model does not run your code: it writes one line of JSON; the harness runs the function.
- ▶ Ill-typed arguments are refused before they run; types are checked, meaning is not.
- ▶ **The description is the only part the model reads when deciding to call it at all.**

Source: Model Context Protocol specification, revision 2026-07-28, tools; Miao & Feng, ch. 15, §4.3.

Step 3: The Whole Loop, in a Dozen Lines

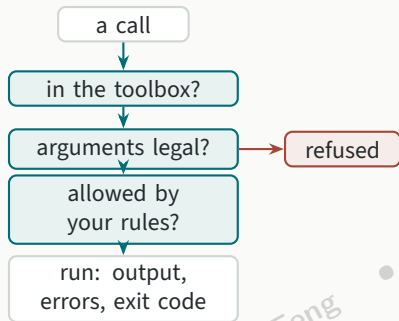
```
messages = [user("Plot uninsurance by age.")]

while True:
    r = model(system=RULES,      # standing rules
              tools=TOOLS,      # names only
              messages=messages) # the whole list
    if r.stop_reason != "tool_use":
        break                  # nothing to run
    messages.append(assistant(r.content))
    out = [run_tool(c) for c in r.content
           if c.is_tool_use]
    messages.append(user(out))
```

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3; Yao et al. (2023).

- ▶ A person writes this file, once; the model never writes or runs it. The literature calls the loop **ReAct**: reason, act, observe.
- ▶ What the model chooses: which tool, which arguments, and when to stop asking.
- ▶ **The list only grows, and is re-sent whole on every pass. That line is your cost curve.**

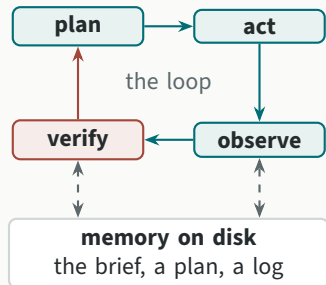
Step 4: Three Rules Around the Loop



- ▶ Fail any one and the call is refused — and the refusal goes back to the model as an observation, which is how it learns the boundary.
- ▶ The lab's loop writes the rules as code: an allow-list, a gate that can block a write, and a budget on turns and tokens.
- ▶ **A syntax line at the head of an ado-file does this job already: an illegal call is stopped before it computes.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3; the session's lab, block A.

Step 5: Memory and a Verifier — Plan, Act, Observe, Verify



- ▶ **Orchestrator**: the loop from Step 3; the model makes every decision in it.
- ▶ **Memory**: files read at the start and written as it goes; the lab's loop writes a table only with its source.
- ▶ **Verifier**: a test, a benchmark or a reviewer that decides what survives.
- ▶ **Persistence and checking complete Deck 6.1's four marks: each is now code or a file.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3; the session's lab, block A.

Run It: Seven Turns, Traced

turn	what it called	what came back
1	rank the statements: inflation, March 2022	three candidate meetings
2	that meeting's sentences on inflation	the quotation, and the statement's URL
3	the yield over the following month	2.32 to 2.89 per cent
4	a shell command, to read the panel itself	refused: not on the allow-list
5	write the answer as a table	blocked: the table names no source
6	the same write, now carrying its source	written
7	the answer: quotation, URL, window, change	end of turn

- **The rules act on two of the 7 turns. The trace is the deliverable; the answer is what it happens to contain.**

Source: The session's lab, block A, the replayed run.

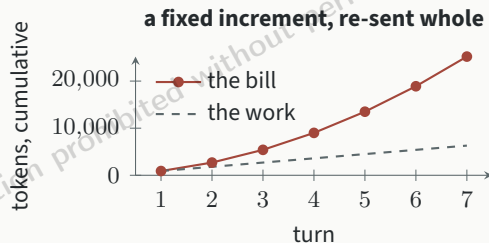
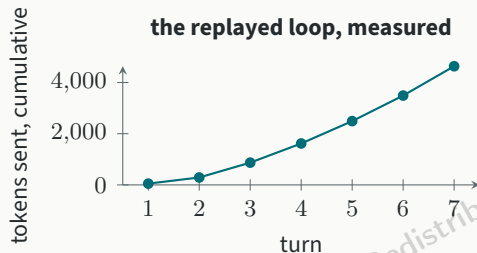
The Session Is a List, Not a Connection



- ▶ The endpoint keeps nothing, so each turn is a fresh request carrying the whole conversation so far.
- ▶ “The window is full” means this list outgrew the limit — and it grows fastest from tool output, not from what you type.
- ▶ **Clearing a session and compacting it are both operations on a list.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §2.1.

The Session Is a List, So the Bill Grows Faster Than the Work

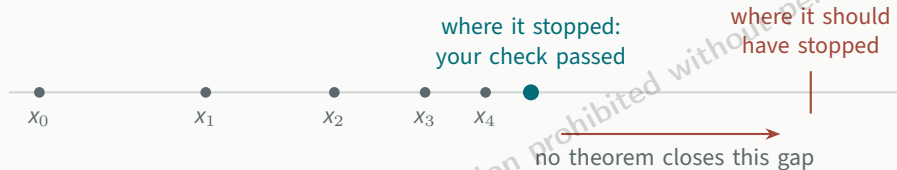


- ▶ A turn appends to the list and re-sends all of it: the work grows by the increment, the bill by the length. Under a budget it can stop mid-task, with no answer.
- ▶ **Sum a list that grows linearly and you get a square: twice the turns, four times the tokens. Files and short sessions bend it back.**

the lab's replayed loop, a fixed script in place of the model, tokens counted at four characters each · built 20 September 2026 ·
the lab's bounded task; exported 23 September 2026

Source: Course-recorded replay, exported by the lab's exporter; Miao & Feng, *AI for Economists*, ch. 15, §3.3.

An Iteration Without a Convergence Theorem



- ▶ The loop is an iteration in the ordinary numerical sense: no closed form, just the newest value used immediately by the next step.
- ▶ Stopping is declared by something testable — a test passes, an exit code is zero, an output matches a benchmark fixed beforehand.
- ▶ **Gauss–Seidel has conditions under which it converges; this loop has none. It can settle, with confidence, on a wrong answer that passes a wrong check.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3, on the loop and its disanalogy.

From Thirty Lines to a Production Harness

in your thirty lines

the message list
the system text
the tools
the allow-list, the gate
the budget
a second loop
a prompt loaded on demand
propose, do not act

in Claude Code (and every harness like it)

the session: resume it, clear it, compact it
the standing brief (CLAUDE.md) and rules, loaded at every start
Read, Edit, Bash, Grep — and servers on one open protocol
permission rules, and hooks that can refuse a call
the meter: context used, tokens spent
a sub-agent, with a list of its own
a skill
plan mode

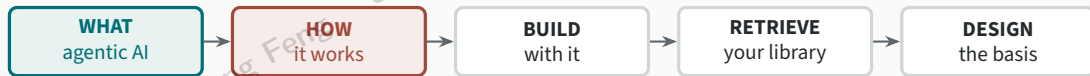
- ▶ The model decides what to run; the harness runs it; R or Stata still does the estimation.
- ▶ **The weights never change. What the harness adds is closed feedback, persistence, and observation over recall.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.2–§3.3, and its thinkbox on what a harness changed; harness command-line help, course machine, 2026-09-20.

Session 6.2

The Production Loop: Install and Drive

The same loop on a laptop and on a login node, and a first hour that ends in a commit



Install Once: Three Durable Steps

```
# 1. install the CLI, once
#   per machine
$ curl -fsSL https://claude.ai/\
    install.sh | bash

# 2. authenticate, once
$ claude
#   a browser opens; sign in

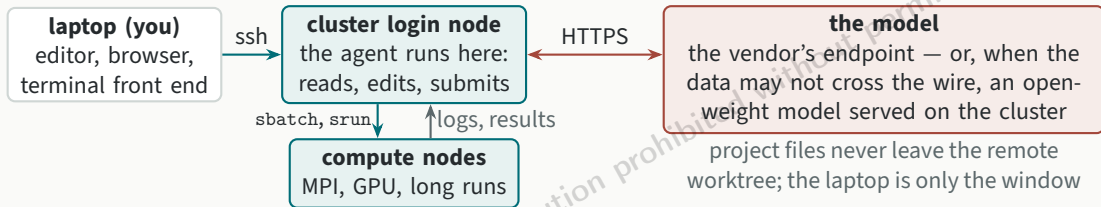
# 3. work from the project,
#   every session
$ cd my-project
$ claude
```

Install once per *machine*. Authenticate once per *account*. Start from the project folder every session — that folder is the agent's whole world.

- ▶ Only step 1 differs between tools: Codex CLI, Antigravity CLI and Qwen Code run the same loop and speak the same tool protocol.
- ▶ **Versions on the course machine on 20 September 2026: 2.1.278, 0.155.1, 0.47.0.**

Source: Vendor installation documentation; versions probed on the course machine, 20 September 2026.

Move the Agent to the Compute: The Cluster Install



- ▶ No `sudo`: node via `nvm` or `module`, the binary in `$HOME/.local/bin`, and sign-in prints a URL you open on the laptop (`claude auth login`, read from its help and not run).
- ▶ **The rule: install where the data, modules, GPUs and job logs live.** A local agent copies cluster files back and forth; fine for toy code, fragile for research.
- ▶ Orchestrate on the login node, compute through the scheduler: centres stop agent processes there “without prior notice”. Work inside `tmux`.

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.2; Anthropic, “Long-running Claude for scientific computing” (23 Mar 2026); Aalto, “AI Agents on HPC”, 23 Sep 2026.

Git Is the Safety Net



- ▶ An agent that can write files can also overwrite them. Version control turns that from a disaster into an inconvenience.
- ▶ Commit before you hand over the keyboard, and again whenever a step worked. `git diff` is the audit: read it before you believe a report.
- ▶ **Without it you are asking a stranger to edit your only copy.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §4.2.

Your First Session: Read First, Plan Before Code

```
$ cd fomc-yields  
$ git add -A && git commit -m wip  
$ claude
```

```
> Read the files here and tell me  
   what this project does. Do not  
   change anything yet.
```

- ▶ The first instruction is read-only on purpose. You are checking whether it understands the project, not whether it can type.
- ▶ Then **plan mode**: it reads and proposes, and writes nothing until you approve. The plan is a file; resume from it, not from the transcript.
- ▶ **Ask it to restate the task in its own words. Disagreement is cheapest here.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §4.2.

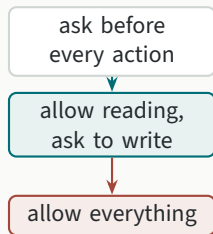
Commands That Earn Their Keep

Command	What it does, and why it earns a place
/init	writes a first standing-instructions file from the repository — the brief re-read at every start
/plan	enables plan mode: it explores and drafts, and changes nothing until you approve
/model	switches model mid-session, context kept: a derivation and a rename do not deserve the same engine
/effort	sets the effort level for the session — the largest quality-against-cost dial you hold
/context	draws context use as a grid: the invisible constraint, made visible
/compact	summarises the session so far — one you choose beats one that fires mid-derivation
/rewind	restores the conversation, or the files, to a checkpoint (Escape twice)

Four dials: what it reads, how hard it thinks, how much it may do unasked, what it costs.

Source: Harness command-line help and installed packages, course machine, 20 September 2026.

Permissions: The Blast-Radius Dial



only in a
container you
can throw away

Where to set it is a rule, not a mood: Deck 6.3 names it — least privilege — and says which three things an agent must never hold at once.

- ▶ Irreversible actions — deleting, sending, spending — are confirmed one at a time.
- ▶ **The dial is the main safety control you have, and it is yours to set per task.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §4.4.

Your First Hour: A Walkthrough Checklist

step	the command	what you look at
1 install	<code>curl -fsSL https://claude.ai/install.sh bash</code>	claude --version answers
2 version control	<code>git init && git add -A && git commit -m "before"</code>	a clean tree to fall back to
3 the brief	claude, then <code>/init</code> ; edit the file it wrote	goal, rules, one gate
4 plan	<code>/plan</code> : “restate the task; propose steps; change nothing”	does it understand the project?
5 first task	one read-only task, then one small write	the transcript
6 read the diff	<code>git diff</code>	every changed line, before any report
7 commit	<code>git add -A && git commit -m "first agent task"</code>	a checkpoint with your name on it
8 the meter	<code>/cost</code> (and <code>/usage</code> for the plan’s meter)	what the hour cost, before the day does

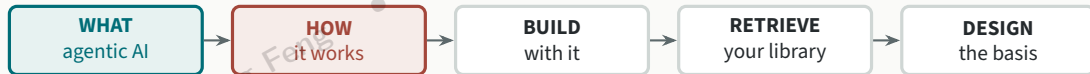
Eight steps, and only 3, 4 and 5 involve the model. The rest is the discipline you already had, done before and after. Steps 2, 6 and 7 were executed in a scratch folder on the build date; the harness commands were read from its help; no agent was launched for this deck.

Source: Course machine, 23 Sep 2026; Miao & Feng, *AI for Economists*, ch. 15, §4.2.

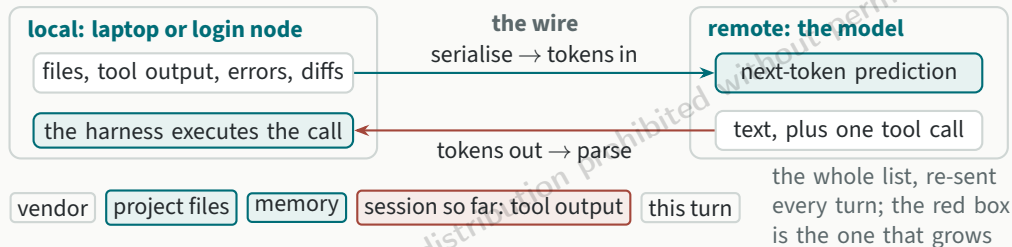
Session 6.2

Inside the Session

What crosses the wire, what fills the window, and what compaction forgets



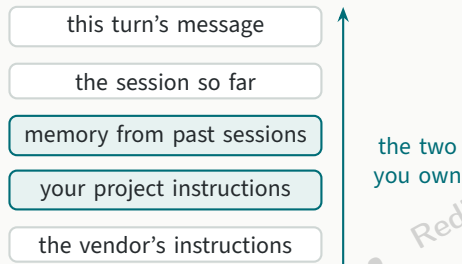
What Crosses the Wire, Drawn With Tokens



- ▶ Nothing of yours moves. The model never logs in; what crosses is text — a tool call out, an observation back — as tokens.
- ▶ Any file it reads crosses too: what it may open is an authorisation decision.
- ▶ **The meter counts both directions, and the part of the list that grows is what your tools printed.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.2–§3.3.

Five Layers of the Context Window

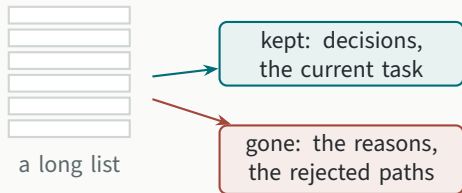


loaded bottom to top,
before you type

- ▶ Every turn, the model decides from one place only: whatever is in this window now.
- ▶ The first layer and the last are not yours to manage. **The middle two are, and both are files on disk.**
- ▶ The window is finite and a long session will exhaust it — but the binding constraint is attention, not size.

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3.

Compaction: What It Keeps, and What It Drops



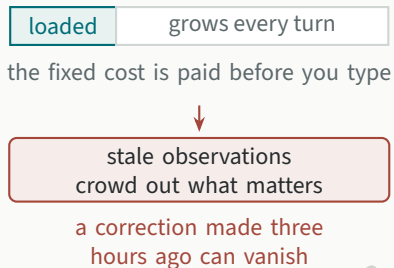
standing instructions and rules
are re-read from disk

- ▶ When the window fills, the harness summarises the session and carries on. It happens on its own unless you do it first.
- ▶ Lossy in a specific way: the conclusions survive; the reasons behind them, and the small corrections you made in passing, often do not.

State belongs in files, not in the conversation. Write down anything that must outlive the window, and compaction costs you nothing.

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3; harness help, course machine, 20 September 2026.

Context Rot, and Why Starting Costs Something



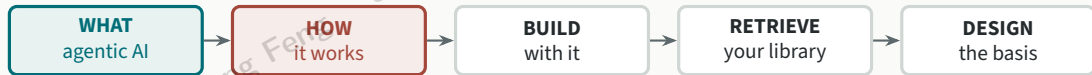
- ▶ **Context rot:** the longer a session runs, the more failed attempts and stale output sit between the model and the thing that matters.
- ▶ The remedy is management, not patience: summarise a finished stage, and start clean for an unrelated task.
- ▶ **Starting is not free either — instructions, rules and memory load first. Short files, short sessions.**

Source: Liu et al. (2024), *TACL* 12; Miao & Feng, *AI for Economists*, ch. 15, §3.3.

Session 6.2

From One Agent to a Team

Where the reasoning lives, how a second loop starts, and the patterns teams are built from



Tool, Skill, Sub-agent: Sorted by Where the Reasoning Lives

tool

code you wrote
the model picks
the *arguments*
no reasoning inside

costs a schema
fails by raising an error

skill

prose you wrote
the model picks *to read it*
reasoning stays
in the main loop

costs one description line
fails by being ignored

sub-agent

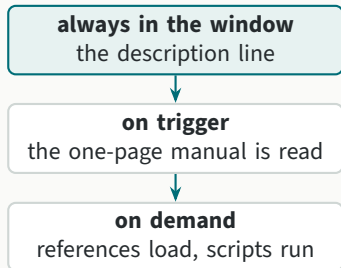
a second loop
its own context and tools
reasoning happens
elsewhere

costs a second context
fails by drifting off-task

- ▶ Loading order is cost order: rules load at every start, a skill's body only when its description matches, a sub-agent in a fresh list.
- ▶ You will read that a skill is “a deterministic piece of software with no brain of its own, which cannot decide when it should be used”. An accurate description — of a tool.
- ▶ **Ask of any new agent term: once it is installed, where does the reasoning happen?**

Source: Miao & Feng, *AI for Economists*, ch. 15, §4.3.

A Skill Is a Folder the Model Opens Only When Needed

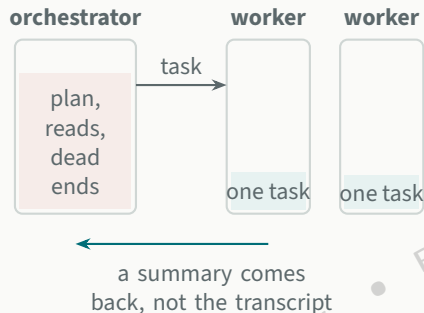


cheap to keep twenty; you
pay for the one you use

- ▶ Three parts: a one-page manual whose description says *when* to call it; scripts for what must be deterministic; references with worked examples.
- ▶ The folder format is an open standard (agentskills.io, published December 2025, originally developed by Anthropic), read by a growing list of agent products — so a lab's skills outlive a vendor.
- ▶ Deck 6.1's meeting loop is one; Deck 6.4 wires another to a real index.
- ▶ **The container is for a team's working knowledge, not for a task.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §4.3; Agent Skills specification and Anthropic's announcement of 18 Dec 2025, read 23 Sep 2026.

Isolated Context, and Dispatch by Description



- ▶ A sub-agent is not extra intelligence — the model is the same. It is a second, empty list.
- ▶ The worker never sees your dead ends, so it cannot be misled by them, and your list grows by a summary rather than by everything it read.
- ▶ Dispatch is a tool call chosen from your descriptions.
- ▶ **A clean window buys focus, not independence: the worker still inherits your framing.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §4.3; Anthropic engineering, multi-agent research systems (13 June 2025).

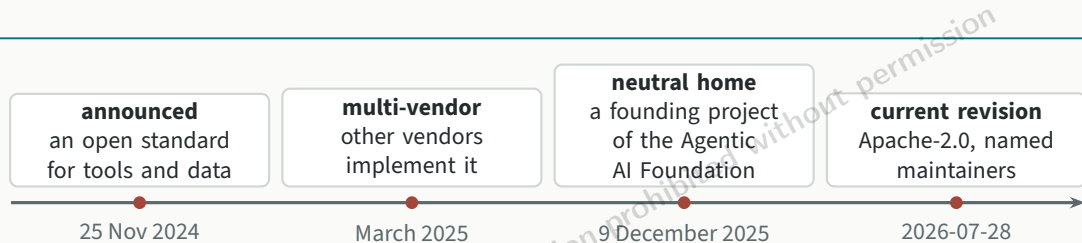
Four Orchestration Patterns, and Which One Claude Code Runs

pattern	shape	in Claude Code
orchestrator-workers	a lead splits the task and dispatches workers	the session dispatching sub-agents
reflection	an output is critiqued and revised before it returns	a reviewer agent; Step 5's verifier
plan, then execute handoff	one pass plans, a second carries it out control passes to a specialised agent	plan mode, then the build a plan file the next session picks up

- ▶ Handoffs “are represented as tools to the LLM”: dispatch is a tool call chosen from your descriptions, and editing one stops it.
- ▶ **The same loop, wired differently. Which wiring your project needs is Deck 6.3’s question.**

Source: Pattern names: Anthropic, “Building effective agents”, 19 Dec 2024 (orchestrator-workers); A. Ng, *The Batch*, 27 Mar 2024; OpenAI Agents SDK docs, handoffs; Claude Code docs, permission modes; all read 24 Sep 2026.

One Protocol for What Lies Past Your Disk



- ▶ Deck 1.3 put “one protocol” on the 2024 mark. This is it: the Model Context Protocol.
- ▶ The neutral home is the Linux Foundation’s **Agentic AI Foundation**, founded with the protocol (from Anthropic), goose (from Block) and AGENTS.md (from OpenAI).
- ▶ So MCP is one of three standards you already use, with the brief file and the skill folder (f25); pin its revision as you pin a package.
- ▶ **Cite the revision, not the vendor: as of 20 September 2026 it is 2026-07-28.**

Source: Anthropic, “Introducing the Model Context Protocol” (25 Nov 2024); Linux Foundation press release, 9 Dec 2025; the protocol’s governance and versioning pages, read 20 September 2026.

What You Now Understand, and What Deck 6.3 Builds With It

tool use
a typed
function

autonomy
the loop
decides

iteration
observations,
refusals

persistence
files on disk

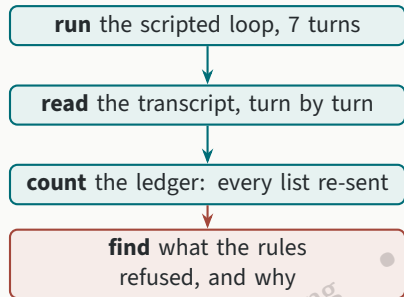
a team
sub-agents,
patterns

checks around all of it: an allow-list, a gate, a budget, a verifier

- ▶ You wrote the loop, ran it offline, and met the same loop in Claude Code, on a laptop and on a login node.
- ▶ You know what one session carries and costs, and how one agent becomes a team.
- ▶ **Deck 6.3 builds with it: a repository that starts itself, a team you can check, and a method for growing a project.**

Source: Miao & Feng, *AI for Economists*, ch. 15, §3.3 and §4.3.

Your Turn: Lab Block A, the Loop Offline



- ▶ Self-paced, after class, offline: a CPU, no account, a clean kernel, fixed seeds. About twenty minutes.
- ▶ The loop's model is a script, not a network call, so your transcript and the recorded one agree line for line.
- ▶ **Block A is Part A of this deck, run: the same classes, the same rules, the same bill.**

Source: The session's lab, block A, and its recorded-results files.